

图论强化训练

环检测专项练习 (Python)

涵盖内容:

- ✓ 图的邻接表表示与构建
- ✓ 无向图 DFS 环检测 (回边 + parent 追踪)
- ✓ 有向图 DFS 环检测 (三色标记法)
- ✓ 有向图 BFS 环检测 (Kahn 算法 + 入度)
- ✓ 非连通图的全局环检测处理

语言: Python 请先仔细阅读背景知识与引导性问题, 再动手编码

第 1 题 无向图中的环检测 (DFS)

考点: DFS 回边 • parent 参数 • visited 集合 • 非连通图处理

1.1 背景知识

无向图中, 环 (cycle) 是指存在一条路径, 使得从某顶点出发, 经过若干条边后又能回到该顶点, 且路径上至少经过 3 个不同的顶点。

在 DFS 遍历中, 除了往子节点前进的“树边”之外, 还可能遇到一种特殊的边——它指向的顶点已经在 visited 集合中。这条边称为回边 (back edge), 是无向图中存在环的充要条件。

难点在于: 无向图的每条边 (u, v) 在邻接表中存储了两次, DFS 从 u 到 v 之后, 在 v 的邻居里会立刻看到 u , 这并不代表存在环, 只是原路返回。因此需要记录当前节点的父节点 (parent) 来排除这种情况。

1.2 引导性问题

在开始编码前, 请先独立思考以下问题。这些问题将直接对应你实现中的关键细节。

1. 请画出一个含有 4 个顶点、含环的无向图, 和一个含有 4 个顶点、不含环的无向图 (树形结构)。DFS 遍历这两张图时, 分别会遍历哪些边?
2. 当 DFS 从顶点 u 出发, 访问邻居 v , 发现 v 已经在 visited 中, 这一定说明存在环吗? 请考虑 v 就是 u 的父节点这种情形。你应该如何修改判断条件来排除这种假阳性?
3. 函数签名 `dfs(node, parent)` 中的 `parent` 参数应该如何初始化? 当图从顶点 0 开始遍历, 没有父节点时, `parent` 传入什么值合适?
4. 如果图是非连通的 (存在多个连通分量), 只从顶点 0 出发做一次 DFS, 能检测到所有连通分量中的环吗? 应该如何修改外层逻辑来解决这个问题?
5. 使用递归实现 DFS 时, 如果图规模很大 (顶点数达到数万), 递归可能触发 Python 的调用栈溢出 (RecursionError)。你能想到一种用显式栈 (list 模拟) 替代递归的方法吗? (选做思考)

1.3 题目要求

实现以下 Python 类中的三个方法:

1. `add_edge(u, v)`: 向无向图中添加一条边, 在邻接表中同时记录 $u \rightarrow v$ 和 $v \rightarrow u$ 。
2. `_dfs_cycle(node, parent, visited)`: 内部辅助函数, 以 `node` 为当前节点、`parent` 为父节点进行递归 DFS, 若发现回边则返回 True。
3. `has_cycle()`: 外层函数, 遍历所有顶点, 对每个未访问的顶点调用 `_dfs_cycle`, 只要任意连通分量含环即返回 True, 否则返回 False。

1.4 接口参考（仅声明，补全实现）

```
1 from typing import List, Set
2 from collections import defaultdict
3
4 class UndirectedGraph:
5     def __init__(self, n: int):
6         """
7         n: 顶点数量, 顶点编号为 0 到 n-1
8         """
9         self.n = n
10        self.adj: dict = defaultdict(list) # 邻接表
11
12    def add_edge(self, u: int, v: int) -> None:
13        """添加无向边 (u, v)"""
14        pass # TODO
15
16    def _dfs_cycle(self, node: int, parent: int,
17                  visited: Set[int]) -> bool:
18        """
19        DFS 辅助函数:
20        - node: 当前正在访问的顶点
21        - parent: 到达 node 时所来自的父顶点 (-1 表示无父顶点)
22        - visited: 已访问顶点的集合
23        返回: 若在以 node 为根的 DFS 子树中发现环则返回 True
24        """
25        pass # TODO
26
27    def has_cycle(self) -> bool:
28        """
29        外层函数: 对所有未访问顶点逐一调用 _dfs_cycle,
30        处理非连通图的情形。
31        返回: 图中是否存在环
32        """
33        pass # TODO
```

1.5 测试用例

```
1 if __name__ == "__main__":
```

```
2  # 情形 1: 无环无向图 (一棵树)
3  # 0 - 1 - 2
4  #   |
5  #   3
6  g1 = UndirectedGraph(4)
7  g1.add_edge(0, 1)
8  g1.add_edge(1, 2)
9  g1.add_edge(1, 3)
10 print(g1.has_cycle())# False
11
12 # 情形 2: 含环无向图
13 # 0 - 1 - 2
14 #  \ /
15 #   \ /
16 #   3 /
17 #  | /
18 #   2<- 即 0-1-2-3-0 构成一个环
19 g2 = UndirectedGraph(4)
20 g2.add_edge(0, 1)
21 g2.add_edge(1, 2)
22 g2.add_edge(2, 3)
23 g2.add_edge(3, 0)
24 print(g2.has_cycle()) # True
25
26 # 情形 3: 非连通图, 仅其中一个连通分量含环
27 # 分量 A: 0 - 1 - 2 (无环)
28 # 分量 B: 3 - 4- 5- 3 (有环)
29 g3 = UndirectedGraph(6)
30 g3.add_edge(0, 1)
31 g3.add_edge(1, 2)
32 g3.add_edge(3, 4)
33 g3.add_edge(4, 5)
34 g3.add_edge(5, 3)
35 print(g3.has_cycle()) # True
36
37 # 情形 4: 单顶点, 无边
38 g4 = UndirectedGraph(1)
39 print(g4.has_cycle()) # False
40
41 # 情形 5: 两个顶点, 一条边 (不构成环)
```

```
42 g5 = UndirectedGraph(2)
43 g5.add_edge(0, 1)
44 print(g5.has_cycle()) # False
```

第 2 题 有向图中的环检测 (DFS 三色标记)

考点: DFS 三色标记 • 递归路径 • 有向图回边 • 非连通处理

2.1 背景知识

有向图中, 无向图的 parent 排除法不再适用。

考虑以下有向图:

$$0 \rightarrow 1 \rightarrow 2 \rightarrow 0$$

这是一个有向环。但下面这种情形没有环:

$$0 \rightarrow 2$$

$$1 \rightarrow 2$$

从 0 出发访问 2, 再从 1 出发时发现 2 已在 visited 中, 但这里并不存在环——2 只是被两条路径共享了。

因此, 有向图需要区分三种状态, 称为三色标记法 (也叫白灰黑标记):

- **WHITE (0)**: 尚未访问。
- **GRAY (1)**: 正在访问中, 当前顶点位于 DFS 递归调用栈上。
- **BLACK (2)**: 已经完全访问完毕, 其所有后继均已处理。

关键结论: 在 DFS 中, 若发现某条有向边 $u \rightarrow v$, 且 v 当前状态为 **GRAY**, 则说明 v 仍在当前递归路径上, 这条边是一条回边 (back edge), 图中存在有向环。若 v 状态为 **BLACK**, 说明 v 已经处理完毕, 该边不会构成环, 可以安全跳过。

2.2 引导性问题

1. 有向图和无向图的环检测最根本的区别是什么? 为什么无向图中有效的 parent 排除法在有向图中会失效? 请举出一个反例。
2. 三色标记中, **GRAY** 代表什么含义? 当你从顶点 u 的邻居中发现一个 **GRAY** 顶点 v 时, 这说明了什么? 能否在图上画出这条“回边”所对应的环?
3. 为什么发现 **BLACK** 顶点不能判定存在环? 请构造一个具体例子, 说明有向图中同一个顶点可以被多条不同路径访问, 却不存在环。

4. DFS 结束时（函数即将返回时），当前顶点的颜色应该从 GRAY 变为什么？如果忘记做这个状态更新，会出现什么错误？
5. 如果有向图有多个连通分量（某些顶点无法从顶点 0 到达），外层循环应该如何处理？与第 1 题相比，逻辑上有何相同之处？

2.3 题目要求

实现以下 Python 类中的三个方法：

1. `add_edge(u, v)`: 向有向图中添加一条有向边 $u \rightarrow v$ ，仅在邻接表中记录 $u \rightarrow v$ 。
2. `_dfs_cycle(node, color)`: 内部辅助函数，以 `node` 为当前节点进行递归 DFS，使用 `color` 列表记录每个顶点的三色状态，若发现回边则返回 `True`。
3. `has_cycle()`: 外层函数，初始化 `color` 列表，遍历所有顶点，对每个 WHITE 顶点调用 `_dfs_cycle`。

2.4 接口参考（仅声明，补全实现）

```
1 from collections import defaultdict
2 from typing import List
3
4 WHITE, GRAY, BLACK = 0, 1, 2
5
6 class DirectedGraph:
7     def __init__(self, n: int):
8         """
9         n: 顶点数量，顶点编号为 0 到 n-1
10        """
11        self.n = n
12        self.adj: dict = defaultdict(list) # 有向邻接表
13
14    def add_edge(self, u: int, v: int) -> None:
15        """添加有向边 u -> v"""
16        pass # TODO
17
18    def _dfs_cycle(self, node: int, color: List[int]) -> bool:
19        """
20        DFS 辅助函数（三色标记）：
21        - node: 当前正在访问的顶点
22        - color: 长度为 n 的列表，color[i] 为顶点 i 的当前颜色
```

```
23     进入时将 node 标记为 GRAY;
24     递归处理所有邻居;
25     返回前将 node 标记为 BLACK。
26     若发现邻居为GRAY 则立即返回 True。
27     """
28     pass # TODO
29
30     def has_cycle(self) -> bool:
31         """
32         外层函数：初始化 color 列表（全部设为 WHITE），
33         遍历所有顶点，对每个 WHITE 顶点调用 _dfs_cycle。
34         """
35         pass # TODO
```

2.5 测试用例

```
1  if __name__ == "__main__":
2      # 情形 1: 有向无环图 (DAG)
3      # 0 -> 1 -> 3
4      # 0 -> 2 -> 3
5      dg1 = DirectedGraph(4)
6      dg1.add_edge(0, 1)
7      dg1.add_edge(0, 2)
8      dg1.add_edge(1, 3)
9      dg1.add_edge(2, 3)
10     print(dg1.has_cycle()) # False
11
12     # 情形 2: 含有向环
13     # 0 -> 1 -> 2 -> 0
14     dg2 = DirectedGraph(3)
15     dg2.add_edge(0, 1)
16     dg2.add_edge(1, 2)
17     dg2.add_edge(2, 0)
18     print(dg2.has_cycle()) # True
19
20     # 情形 3: 有向图，共享节点但无环
21     # 0 -> 2
22     # 1 -> 2
23     # 验证：2 被访问两次，但不构成环
```

```
24 dg3 = DirectedGraph(3)
25 dg3.add_edge(0, 2)
26 dg3.add_edge(1, 2)
27 print(dg3.has_cycle()) # False
28
29 # 情形 4: 自环 (顶点指向自身)
30 dg4 = DirectedGraph(3)
31 dg4.add_edge(0, 1)
32 dg4.add_edge(1, 1) # 自环
33 print(dg4.has_cycle()) # True
34
35 # 情形 5: 多个连通分量, 仅其中一个含环
36 # 分量 A: 0 -> 1 -> 2 (无环)
37 # 分量 B: 3 -> 4 -> 3 (有环)
38 dg5 = DirectedGraph(5)
39 dg5.add_edge(0, 1)
40 dg5.add_edge(1, 2)
41 dg5.add_edge(3, 4)
42 dg5.add_edge(4, 3)
43 print(dg5.has_cycle()) # True
```

第 3 题 有向图中的环检测 (BFS + Kahn 算法)

考点: 拓扑排序 • 入度 • Kahn 算法 • 处理顶点数验证

3.1 背景知识

在拓扑排序的讲解中已经提到:

有向图能够完成拓扑排序 $\iff$ 图中不存在有向环

Kahn 算法的核心是不断将入度为 0 的顶点加入队列, 将其从图中”移除” (即将邻居的入度减 1), 再将新出现的入度为 0 的顶点继续入队。

算法结束后:

- 若所有顶点都被处理 (处理数 = 总顶点数 n): 图无环。
- 若处理数 $< n$: 剩余顶点构成至少一个有向环, 它们的入度永远无法降为 0。

这种方法相比 DFS 三色标记更加**直观**, 且天然就是迭代 (非递归) 的, 不会触发 Python 的递归深度限制。

3.2 引导性问题

1. 什么是顶点的入度？请在一张有 5 个顶点的有向图上，手动计算每个顶点的入度，并找出初始时入度为 0 的顶点。
2. 如果图中存在一个有向环 $A \rightarrow B \rightarrow C \rightarrow A$ ，这三个顶点的入度各是多少？在 Kahn 算法执行过程中，它们的入度会变成 0 吗？为什么？
3. 建图时需要同时维护邻接表和入度数组。每调用一次 `add_edge(u, v)`，应该更新哪个顶点的入度？是 u 、 v ，还是两者都需要？
4. 算法开始时，如何初始化队列？遍历完每个出队顶点后，具体应该执行哪些操作？
5. Kahn 算法与 DFS 三色标记相比，各有哪些优缺点？在什么场景下你会优先选择 Kahn 算法？

3.3 题目要求

实现以下 Python 类中的两个方法：

1. `add_edge(u, v)`：向有向图中添加一条有向边 $u \rightarrow v$ ，同时更新 `indegree` 数组。
2. `has_cycle()`：使用 Kahn 算法 (BFS) 检测有向环。初始化队列，执行 BFS，最终通过处理的顶点数判断是否存在环。

3.4 接口参考（仅声明，补全实现）

```
1 from collections import defaultdict, deque
2 from typing import List
3
4 class DirectedGraphBFS:
5     def __init__(self, n: int):
6         """
7         n: 顶点数量，顶点编号为 0 到 n-1
8         """
9         self.n = n
10        self.adj: dict = defaultdict(list) # 有向邻接表
11        self.indegree: List[int] = [0] * n # 每个顶点的入度
12
13    def add_edge(self, u: int, v: int) -> None:
14        """
15        添加有向边 u -> v。
16        注意：需要同时更新 indegree。
```

```
17     """
18     pass # TODO
19
20     def has_cycle(self) -> bool:
21         """
22         使用 Kahn 算法 (BFS) 检测有向环:
23         1. 将所有入度为 0 的顶点加入队列。
24         2. 逐一出队, 将其邻居的入度减 1,
25            若邻居入度降为 0 则入队。
26         3. 统计处理的顶点总数,
27            若等于 n 则无环, 否则有环。
28
29         注意: has_cycle 可能被多次调用,
30         每次调用时请使用 indegree 的副本,
31         避免修改原始数据。
32         """
33         pass # TODO
```

3.5 测试用例

```
1 if __name__ == "__main__":
2     # 情形 1: 有向无环图 (与第 2 题情形 1 相同, 结果应一致)
3     bg1 = DirectedGraphBFS(4)
4     bg1.add_edge(0, 1)
5     bg1.add_edge(0, 2)
6     bg1.add_edge(1, 3)
7     bg1.add_edge(2, 3)
8     print(bg1.has_cycle()) # False
9
10    # 情形 2: 含有向环
11    bg2 = DirectedGraphBFS(3)
12    bg2.add_edge(0, 1)
13    bg2.add_edge(1, 2)
14    bg2.add_edge(2, 0)
15    print(bg2.has_cycle()) # True
16
17    # 情形 3: 自环
18    bg3 = DirectedGraphBFS(2)
19    bg3.add_edge(0, 1)
```

```
20     bg3.add_edge(1, 1)
21     print(bg3.has_cycle()) # True
22
23     # 情形 4: 多个连通分量, 仅其中一个含环
24     bg4 = DirectedGraphBFS(5)
25     bg4.add_edge(0, 1)
26     bg4.add_edge(1, 2)
27     bg4.add_edge(3, 4)
28     bg4.add_edge(4, 3)
29     print(bg4.has_cycle()) # True
30
31     # 情形 5: has_cycle 被多次调用, 图结构不应改变
32     bg5 = DirectedGraphBFS(3)
33     bg5.add_edge(0, 1)
34     bg5.add_edge(1, 2)
35     print(bg5.has_cycle()) # False
36     print(bg5.has_cycle()) # False (第二次调用结果应相同)
```

附：编写建议与常见误区

1. 先从第 1 题开始。无向图 DFS 是最简单的情形，帮助你熟悉图的构建（邻接表）和基本 DFS 框架，再推进到有向图。
2. 无向图不要忘记 `parent` 参数。最常见的错误是，访问到父节点时误判为发现了环。仔细检查你的判断条件：`if neighbor != parent and neighbor in visited`。
3. 有向图不要用 `visited` 布尔值替代三色标记。`visited` 集合无法区分“当前递归路径上”与“已完全处理”，会导致情形 3（共享节点无环）中出现误判。
4. Kahn 算法中 `has_cycle` 应使用入度副本。`indegree[:]` 或 `list(self.indegree)` 都可以得到副本，避免重复调用时数据被污染。
5. 非连通图处理。第 1 题和第 2 题的外层循环要覆盖所有顶点，不能只从顶点 0 出发；Kahn 算法天然能处理非连通图（入度为 0 的顶点来自所有分量）。
6. 自环是最小的有向环。顶点 v 有自环意味着 $v \rightarrow v$ ，三色标记中 v 被标记为 GRAY 后，立刻发现邻居 v 也是 GRAY，直接触发返回。Kahn 算法中， v 的入度至少为 1（来自自身那条边），所以 v 永远无法入队，自然被正确识别为环的一部分。
7. 运行顺序建议。先跑每道题的测试用例，验证所有情形均通过，再尝试比较三种方法在相同输入下的结果是否一致，用此交叉验证你的实现。